

Snap'N'Go: An Extendable Framework for Evaluating Mechanisms in Spatial Crowdsourcing

Sofia Kobayashi
Wellesley College'24
Wellesley, MA, USA
sofia.kobayashi@wellesley.edu

Christine Bassem
Wellesley College
Wellesley, MA, USA
cbassem@wellesley.edu

Yuehe Mao
Wellesley College'24
Wellesley, MA, USA
yuehe.mao@wellesley.edu

Catherine Grevet Delcourt
Wellesley College
Wellesley, MA, USA
cdelcourt@wellesley.edu

ABSTRACT

Given the challenges of evaluating the effect of task allocation mechanisms and their associated incentives on the behavior of participants in spatial crowdsourcing, there is a need for an evaluation framework that simplifies the process of benchmarking such mechanisms with real crowds. In this demo, we present Snap'N'Go, a spatial crowdsourcing application that is modeled as a scavenger-hunt game, which is integrated within a larger modular framework that can be easily extended to evaluate various task allocation mechanisms and incentive models, without affecting the user-experience. Moreover, we present the details of the initial launch of the framework, in which we benchmark multiple mechanisms, and discuss how it can be easily extended for various experimental settings.

CCS CONCEPTS

• **Information systems** → **Spatial-temporal systems**; *Collaborative and social computing systems and tools*.

KEYWORDS

spatial crowdsourcing, incentives, evaluation, piggyback, prototyping

ACM Reference Format:

Sofia Kobayashi, Yuehe Mao, Christine Bassem, and Catherine Grevet Delcourt. 2023. Snap'N'Go: An Extendable Framework for Evaluating Mechanisms in Spatial Crowdsourcing. In *The 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23 Adjunct)*, October 29–November 01, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3586182.3615794>

1 INTRODUCTION

Background and Motivation. Spatial Crowdsourcing has been increasing in popularity in the recent years, in which the mobility of self-motivated crowds is leveraged to assist in completing tasks with varying spatial and temporal requirements. Various citizen-based

sensing platforms [2, 3] fall under this model of crowdsourcing, as well as ride sharing [6, 9] and food delivery [7] platforms. A major challenge in designing task allocation mechanisms in such platforms is the evaluation of the effects of recommendations and their associated incentives on the behavior of the crowds and the performance of the platform as whole [10].

Unlike commercial entities, which have access to volumes of data and participants allowing their constant development of mechanisms [7, 9], researchers typically have to resort to implementing their own small-scale crowdsourcing applications [1, 8] to involve real crowds in the evaluation of their mechanisms. That process is time-consuming and unnecessarily complex, deterring the focus on mechanism evaluation and lacing it with the additional challenges of launching new applications [5].

We argue that there is a need for an open-source, re-usable, and extendable spatial crowdsourcing mechanism evaluation framework, to enable mechanism designers to quickly prototype their mechanisms, and efficiently evaluate their effect on crowd behavior. Thus, breaking down some of the barriers around innovation and benchmarking within this domain.

Contribution. In this work, we develop a modular, extendable, and light-weight spatial crowdsourcing framework designed for evaluating novel task allocation mechanisms and their effect on crowd behavior in realistic settings, namely the Snap'N'Go. To allow for the benchmarking of different mechanisms without affecting the overall user experience, the framework is implemented as a scavenger-hunt game application, allowing clients¹ to modify the allocation/matching and incentives seamlessly. Moreover, the framework is designed to piggyback on various social platforms [4], and its configuration settings can be easily modified to be set up in any spatial setting.

In this demo, we present the Snap'N'Go framework; defining its architecture and highlighting its design features. We also describe the typical workflow of a user on the application, and how easily they can complete tasks. Finally, we present the results of our initial launch of the framework on the Wellesley College campus, in which we benchmarked two allocation mechanisms and two incentive models.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST '23 Adjunct, October 29–November 01, 2023, San Francisco, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0096-5/23/10.

<https://doi.org/10.1145/3586182.3615794>

¹In this paper, we differentiate between the notion of a client and a user. A client is a researcher who would leverage Snap'N'Go in their experiments, and a user is a participant in that experiment.

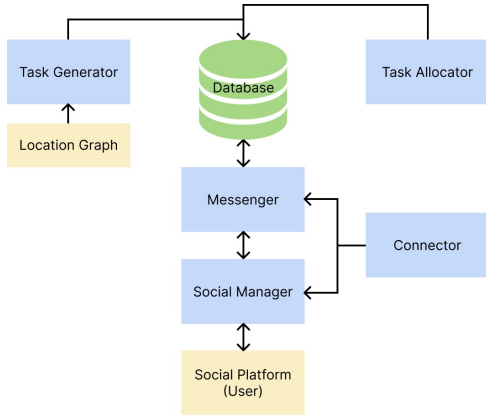


Figure 1: The various modules of Snap'N'Go are designed to interact via the central database, to allow for seamless integration of new mechanisms and social platforms.

2 DESIGN AND FEATURES

Snap'N'Go is a spatial crowdsourcing application that leverages piggyback prototyping to crowdsource the process of taking pictures of various locations and times, similar to a scavenger hunt game. In the current iteration of development, tasks are generated by the framework, and participants are Wellesley College students moving around the campus. Designed with modularity and extendability in mind, the architecture of our proposed framework, as shown in Figure 1, is composed of four main modules and a central database. A main feature of Snap'N'Go is that each of its main modules is abstracted from the others, and any one can be switched out without interfering with the others.

The database stores all data pertinent to tasks generated, participants, and the interaction of the participants with the tasks. It is a simple relational database with three tables, and it acts as the central hub of the framework, allowing the other framework modules to share data without communicating directly with one another, which we describe briefly below.

(1) Social Piggyback Manager. Snap'N'Go can be launched atop existing social platforms, such as Slack and Twitter. To allow for this versatility, this module is composed of two parts; (1) The bot, which handles all interaction with the chosen social media platform using platform-specific code and tools to present assignments to the user and get user input. The current implementation is a Slack Bot, but it can easily be re-implemented to serve a different platform. (2) The messenger handles all interaction between the bot and the database. It both fetches information (e. g. new assignments and user account status) from the database for the bot to make further actions, and also updates the database (e. g. update assignment status and add user submissions to tasks) upon requests from bot. This design allows clients to switch out either the database or the social platform and platform-specific bot without interference between one another.

(2) Task Generator. The task generator defines task parameters, creates new tasks, and inserts them into the tasks table, ready to be assigned to users. A task is defined by its start time, time window, compensation, and description. The first three are generated based

on parameters defined by the client, while the description consolidates that information together with instructions to the user on how to finish the task. Both the location and action can be read from simple json or txt files, so clients can easily create and load in their own graph and action files, allowing for increased flexibility and applicability.

(3) Task Allocation. This module reads from the database to get the available tasks and users, matches tasks to users using a designated matching algorithm, and inserts the task-user assignments into the database to be passed to the bot later. This module is extendable, in that it is designed to receive the designated matching algorithm as a parameter, so clients can define different matching algorithms and easily switch between them when using Snap'N'Go.

(4) Module Connector. This module powers the task-to-user part of Snap'N'Go by administering periodic operation cycles making scheduled calls to each of the other main modules: generating tasks, assigning them to users, then eventually sending the assignments to users through the chosen social media platform. Each of these calls happen on periodic timers which can be detailed by the client; other variables like number of tasks generated per cycle and time limits can also be modified.

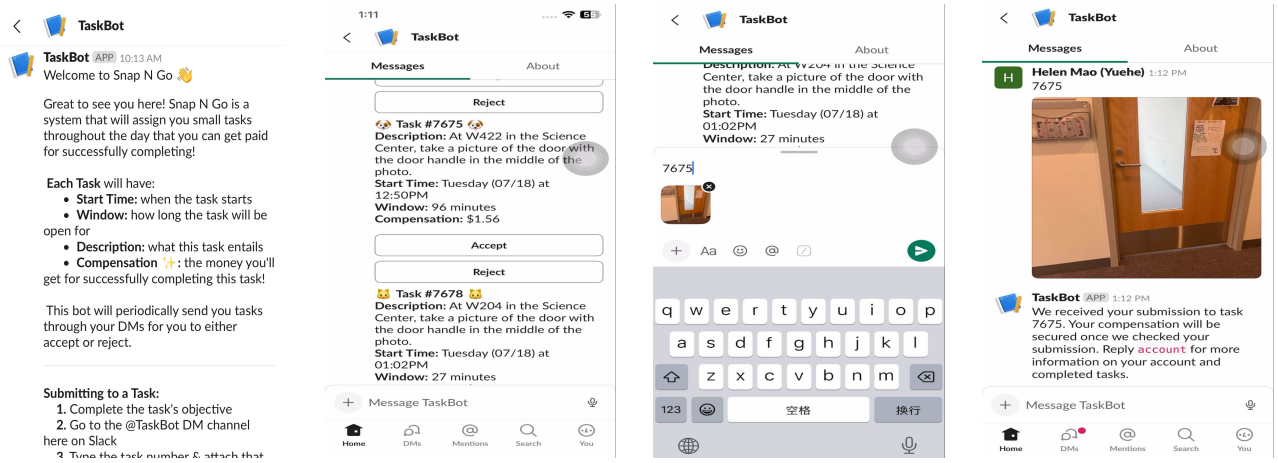
3 INITIAL LAUNCH AND PRELIMINARY RESULTS

An initial version of the application has been launched to test our assumptions on user behaviors. The application was set in a single building on the Wellesley College campus, and has been running for two week, at the point of submitting this abstract, and we will continue running it and collecting results throughout the Fall semester. Random allocation algorithm has been tested with different incentive hypotheses: raised compensations, increased number of tasks, and encouragement on user interaction. We are also currently testing a weighted random allocation algorithm and planning for the addition of more.

Configuring the framework. Location files (json format) are created to describe the various locations within the building in which the experiment is set, and it is accompanied by action files (json format) containing different instructions on the action to be performed while taking a picture at that location (e.g. hold out two fingers or cover half of the door). Then, task descriptions are generated randomly by combining location and actions.

Recruiting Participants. The research team reached out to students working within the building, and 22 Wellesley students signed up for the study. They were all added manually to the Slack channel dedicated to the Snap'N'Go application. Once they join, they are greeted by the framework's bot and given instructions on how to participate, as shown in Figure 2a. Then, throughout the day, the users are assigned tasks, as shown in Figure 2b. User participation is voluntary and users are compensated for interacting with the application, with larger compensations provided for completed tasks.

Mechanisms implemented. For our initial launch, we implemented two matching algorithms. (1) *Random Matching*, in which users and tasks are paired one-to-one completely at random. We use this as our baseline mechanism. Then we tested (2) *Weighted Random Matching*, in which the number of tasks allocated to a user



(a) Welcome message to each user. (b) Tasks allocated directly to user.

(c) User submitting a task.

(d) Final confirmation from bot.

Figure 2: Sample interactions between the Snap'N'Go application and a single user.

depends on the user's reliability, *i.e.*, the ratio of tasks a user has completed to the number they've accepted. A user with higher reliability has a higher chance of being allocated more tasks.

We also modified the incentive model three times; (1) raising the prices of all tasks by more than 100%, (2) increasing the number of tasks generated and allowing for more tasks being available for participants, (3) compensating participants for even rejecting tasks to encourage them to interact with the framework in general.

Preliminary interaction results. In the two week launch, among the 22 students signed up, 11 have interacted with the Slack bot and 8 are active (have been finishing tasks regularly). 71 tasks have been completed so far, averaging about 5 tasks per day. Among these tasks, more than half of them are completed within a few minutes of another submission from the same user, implying users tend to finish more tasks that have overlapping time windows.

In terms of the incentive models, (1) *raising compensation* didn't result in any changes; (2) *increasing number of tasks* resulted in increased number of submissions from three users (38% of active users); (3) *encouraging user interaction by compensating task rejection* resulted in two new active users (33% increase of active users) after receiving some compensation through rejecting tasks. As for the effects of the different allocation algorithms on user behavior, we are in the process of collecting data and compiling results.

On the clients' side, we found that switching to new algorithms is fast and easy under this framework, taking barely any longer than writing the algorithm itself, which is several minutes to a few hours, at most.

4 DISCUSSION AND FUTURE WORK

In this demo, we presented Snap'N'Go, a novel modular and extendable spatial crowdsourcing framework for evaluating task allocation mechanisms and their associated incentive models with real crowds. At the time of this submission, the application had been running successfully for two weeks, allowing for the addition of 2 task allocation algorithms and 3 incentive models.

As this project is still ongoing, we plan to continue to extend the framework with more mechanisms, and conduct longer experiments to better assess the effect of these mechanisms on crowd behavior. Moreover, in order to streamline the process of compensating users, we plan to integrate an online payment service. Lastly, in order to collect qualitative feedback from users, we plan to incorporate mini interactions with users within the application to get quick feedback.

REFERENCES

- [1] G. Bajaj and P. Singh. 2018. Load-Balanced Task Allocation for Improved System Lifetime in Mobile Crowdsensing. In *2018 19th IEEE International Conference on Mobile Data Management (MDM)*. 227–232.
- [2] Raghu K Ganti, Fan Ye, and Hui Lei. 2011. Mobile crowdsensing: current state and future challenges. *IEEE communications Magazine* 49, 11 (2011), 32–39.
- [3] Michael F Goodchild. 2007. Citizens as voluntary sensors: spatial data infrastructure in the world of Web 2.0. *International journal of spatial data infrastructures research* 2, 2 (2007), 24–32.
- [4] Catherine Grevet and Eric Gilbert. 2015. Piggyback prototyping: Using existing, large-scale social computing systems to prototype new ones. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. 4047–4056.
- [5] Jonathan Grudin. 1994. Groupware and social dynamics: Eight challenges for developers. *Commun. ACM* 37, 1 (1994), 92–105.
- [6] Christopher Jonathan and Mohamed F Mokbel. 2017. Towards a unified spatial crowdsourcing platform. In *International Symposium on Spatial and Temporal Databases*. Springer, 379–383.
- [7] Manas Joshi, Arshdeep Singh, Sayan Ranu, Amitabha Bagchi, Priyank Karia, and Puneet Kala. 2022. FoodMatch: Batching and Matching for Food Delivery in Dynamic Road Networks. *ACM Transactions on Spatial Algorithms and Systems (TSAS)* 8, 1 (2022), 1–25.
- [8] Ryoma Kawajiri, Masamichi Shimosaka, and Hisashi Kashima. 2014. Steered crowdsensing: incentive design towards quality-oriented place-centric crowdsensing. In *Proceedings of the 2014 ACM International Joint Conference on Pervasive and Ubiquitous Computing*. ACM, 691–701.
- [9] Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. 2020. Ride-hailing order dispatching at didi via reinforcement learning. *INFORMS Journal on Applied Analytics* 50, 5 (2020), 272–286.
- [10] Yongxin Tong, Zimu Zhou, Yuxiang Zeng, Lei Chen, and Cyrus Shahabi. 2020. Spatial crowdsourcing: a survey. *The VLDB Journal* 29 (2020), 217–250.